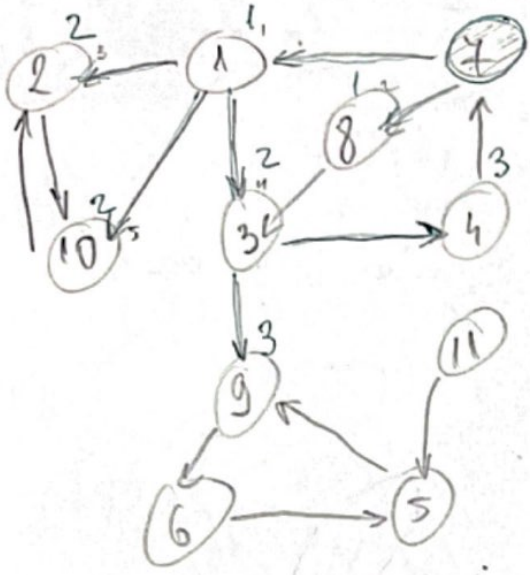
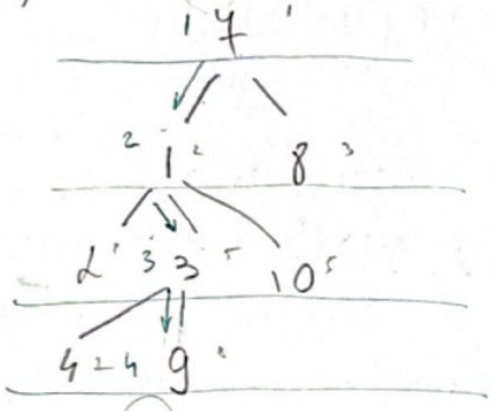
EXEMPLIFICĂRI BF / DF



BF - aplicabil pt drumuri de lung. minime

DF - df forests (desc)
- gasirea unui ciclu

(1) BF(7)



Q: 7, 1, 8, 2, 3, 10, 4, 9

→ Adaug 7 în coadă

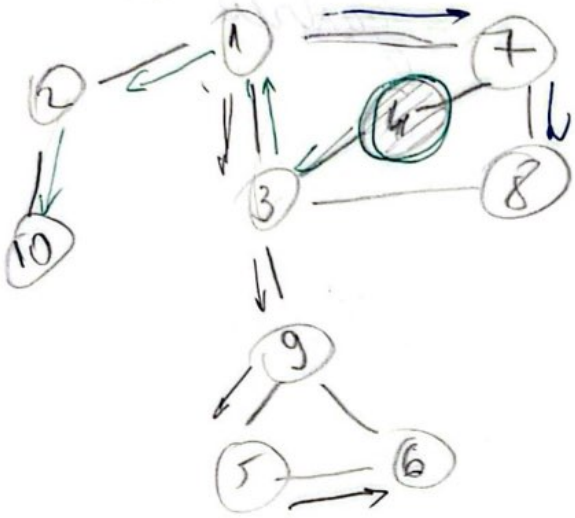
→ La cate - nu pas sunt un nod din coadă, îi parcurg vecinii NEVIZITAȚI și îi adaug în coadă

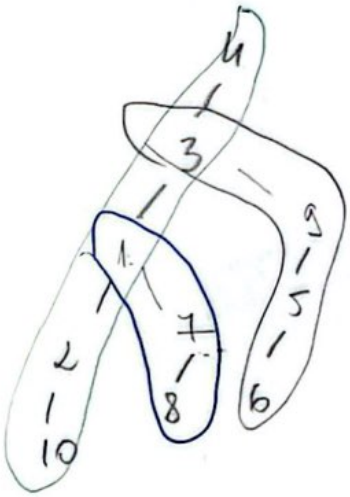
Q_0: 7
Q_1: ~~7~~ 1, 8
Q_2: ~~1~~, 8, 2, 3, 10
Q_3: ~~8~~, 2, 3, 10
Q_4: ~~2~~, 3, 10
Q_5: 3, 10
Q_6: ~~3~~, 10, 4, 9
Q_7: 10, 4, 9
Q_8: ~~4~~, 9
Q_9: ~~9~~
Q_{10}: □

Scanned with CamScanner

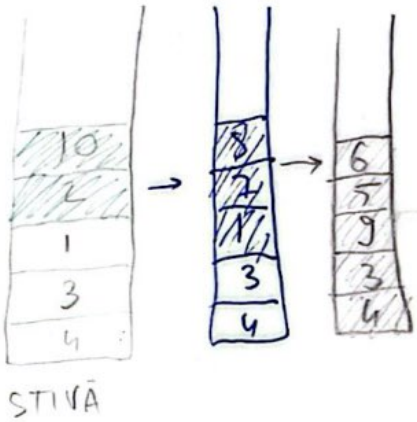
→ Modul în care s-a parcurs folosind BF(7) în cadrul căruia distanța de la 7 la 9 este pt a determina nr minim de muchii ce trebuie parcurse pt a ajunge de la nodul 7 la 9. (M)

(2) DF(4)



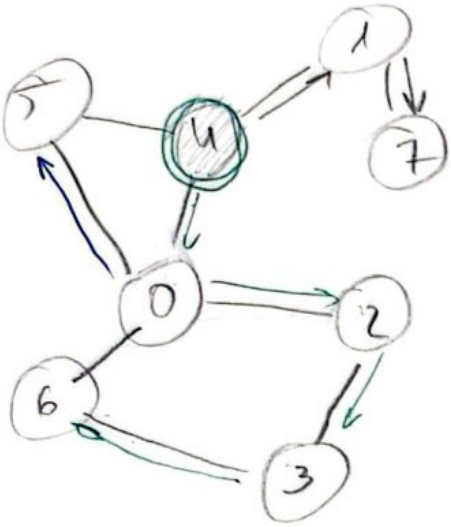


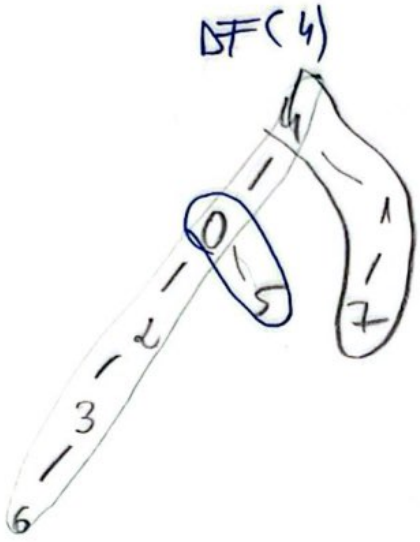
S: 4, 3, 1, 2, 10, 7, 8, 9, 5, 6



→ Punem nodul 4 în stivă
→ Din fiecare nod NEVIZITAT, apelăm pt fiecare vecin NEVIZITAT DF(vecin), morțând până când nu mai avem posibilitatea de a avansa.

(3)





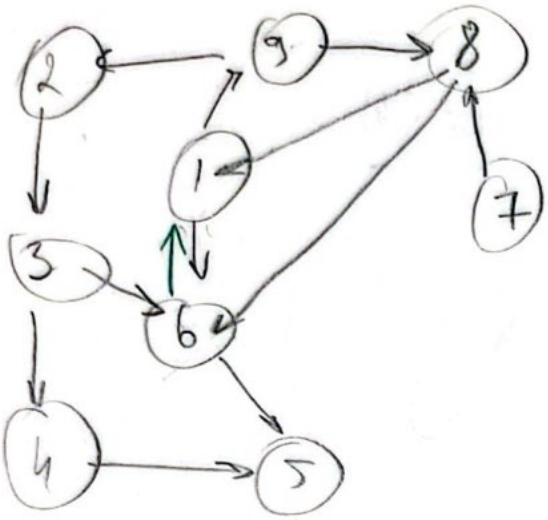
→ începem explorarea dintr-un nod nevizitat (4 în acest caz)
→ căutăm un vecin nevizitat, devenind nod curent.
→ atât timp nodul curent are un vecin nevizitat, repetăm pasul de mai sus (apelăm DF)
→ când nodul curent nu mai are vecini nevizitați, ne întoarcem ca strămoși până când acesta are vecini nevizitați.

COMPONENTE TARE CONEXE (CTC)
↳ Tarjan
↳ Kosaraju

(*) ex graf:

→

Scanned with CamScanner



COMP. CONEXE

$C_1: 7$ $C_4: 6$ $C_7: [9, 8, 1]$

$C_2: 5$ $C_5: 3$

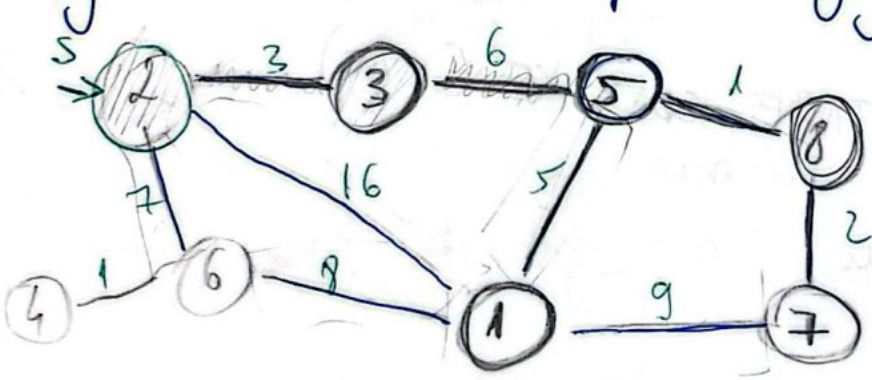
$C_3: 4$ $C_6: 2$

⇒ Schimbam sens arc pt a crea o comp conexă mai mare ⇒ $(1,6) \Rightarrow (6,1)$

⇒ putem avea comp. $[2, 3, 6, 9, 8, 1]$

EXEMPLIFICARE ALGORITMI

- Algoritmul lui Prim pentru graful:



→ PAȘI

- pornim din nodul 2.

! La Prim pornesc mereu de la nodul de start

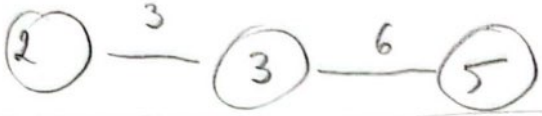
Scanned with CamScanner

= 2 (2) (verificăm costul muchiei/or vecine/or să vedem care este mai mic)

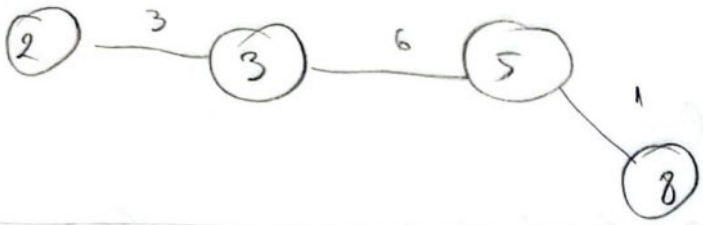
(1)



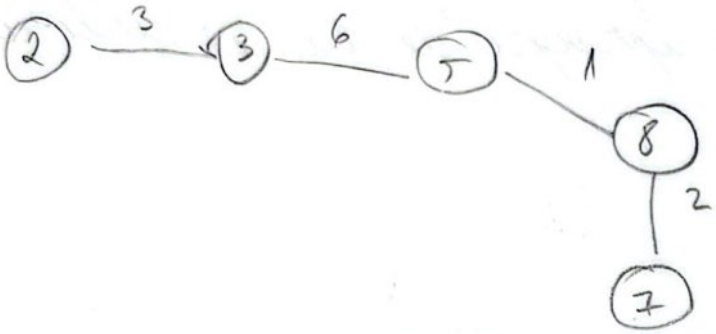
(2)



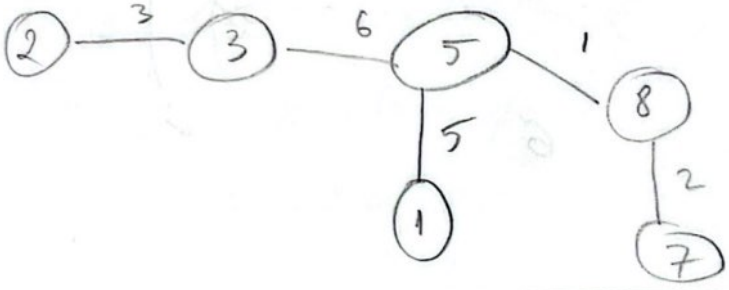
(3)



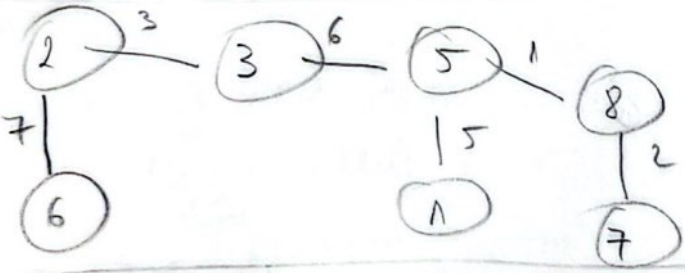
(4)



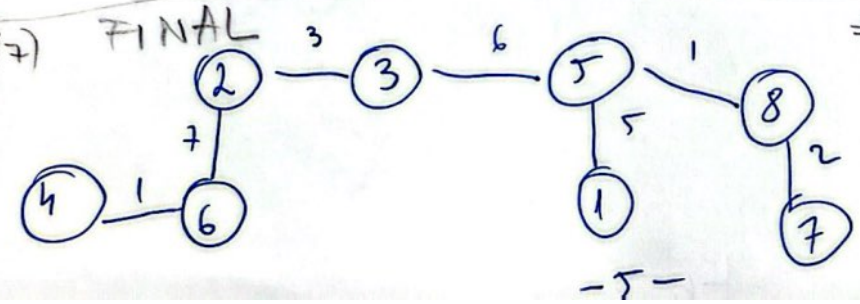
(5)



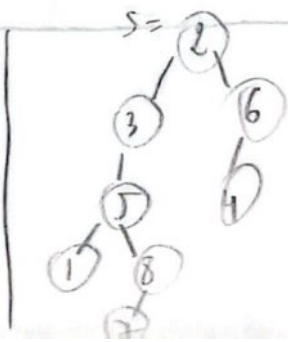
(6)



(7) FINAL



⇒

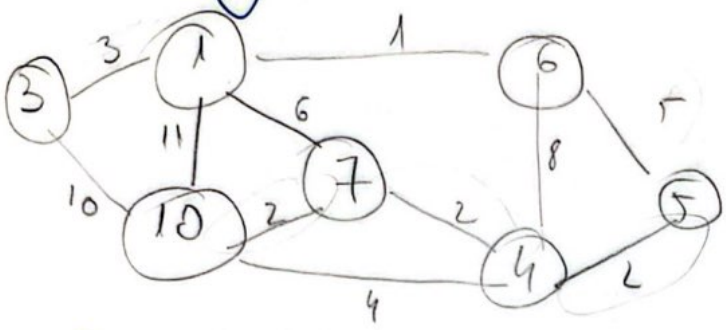


-5-

EXEMPLIFICARE KRUSKAL.

! LA APCM NU FOR CICLURI !!!

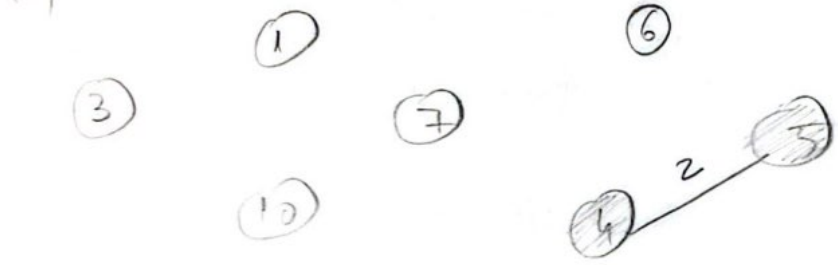
Pentru graful



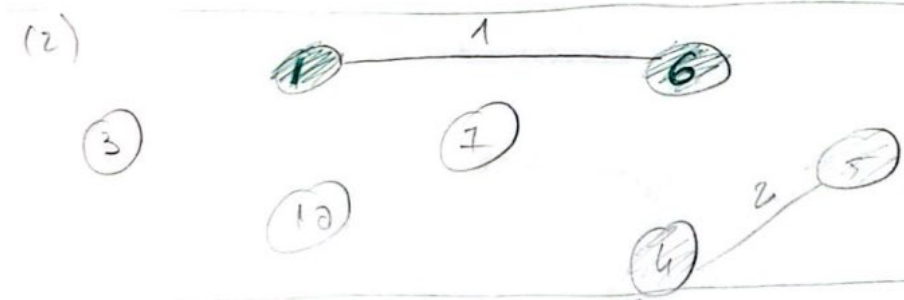
→ Pași Kruskal:

! nodurile vor fi izolate și vom face legăturile dar aproape în aproape de la cel mai mic cost.

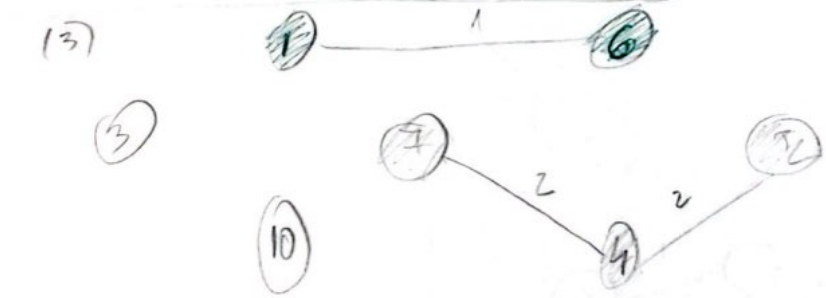
(1)



(2)

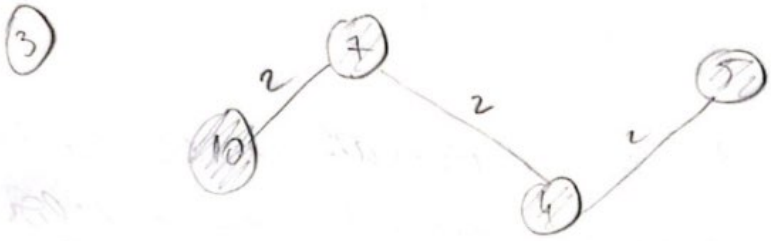


(3)

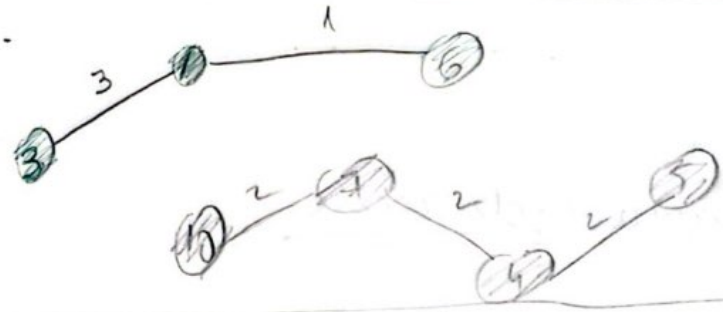


Scanned with CamScanner

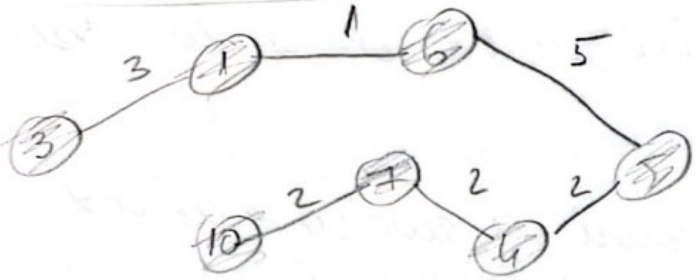




(7)



(8)



FINAL

□

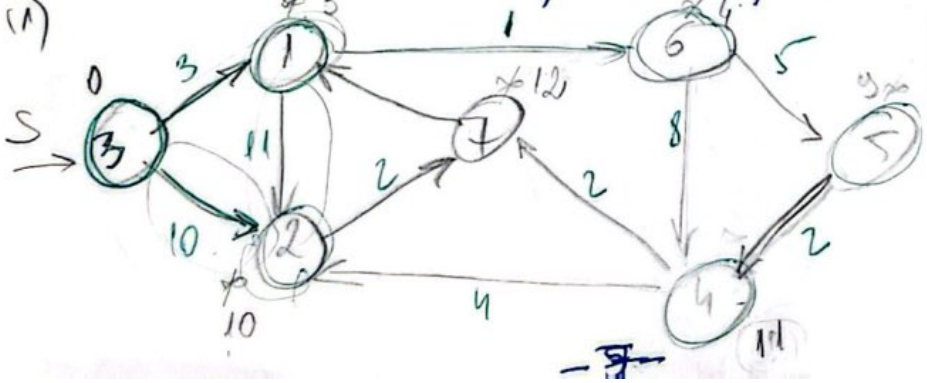
EXEMPLIFICARE DIJKSTRA

- Algoritmi pentru grafuri orientate cu circuite și ponderi POZITIVE ⇒ DIJKSTRA

→ PASI:

1) (x) Ne vom folosi de un tabel în care notăm nodul curent și estimația.

(1)



1	2	3	4	5	6	7
∞/0	∞/0	0/0	∞/0	∞/0	∞/0	∞/0

→ Dijkstra x după nu atta capete! (-8-)

EXEMPLIFICARE BELLMAN FORD.

· Algoritm pentru grafuri orientate cu circuite și ponderi reale, detectând existența circuitelor negative → B.Ford.

→ PASI:

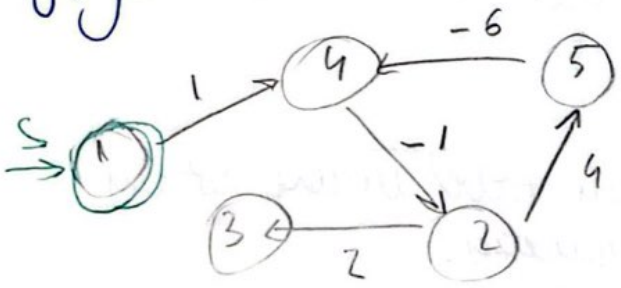
* la un pas relaxăm toate arcele

⊛ B.Fr. îți oferă cea mai scurtă cale de la nodul x la nodul y.

⊛ B.Fr. lucrează pe grafuri cu muchii cu cost negativ

B.Fr. ≠ Greedy

fie graful



S = 1

$E = \{(1,4), (2,3), (4,2), (2,T), (T,4)\}$

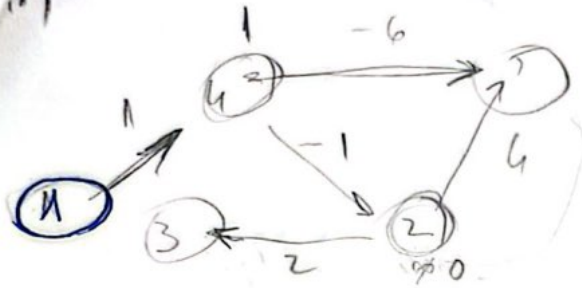
vector de elem. [0, 9, -4, -11, -5]

Scanned with CamScanner

5 noduri ⇒ 4 iterații

selectăm 1 (luăm lexicogr.)

1)

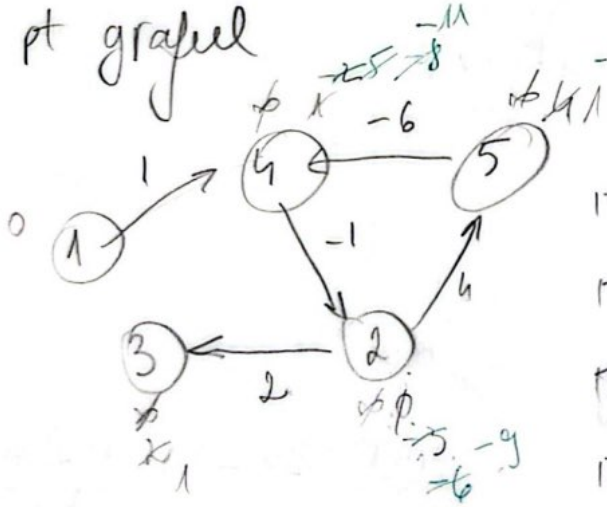


	1	2	3	4	5
	0	∞	∞	∅	∞
IT. 1	0	0/4	∅	1/1	-5/4
IT. 2	0	0/4	2/2	1/1	-5/4
IT. 3	0	0/4	2/2	1/1	-5/4
IT. 4	0	0/4	2/2	1/1	-5/4

→ pentru a determina dacă există un ciclu cu cost negativ accesibil din S, se mai face o iterație suplimentară în care se verifică dacă mai există o muchie care poate fi relaxată.

→ Dacă în acea iterație se găsește o muchie (u, v) pt. care d[u] + w(u, v) < d[v], atunci există un cost negativ în graf.

pt graful



	1	2	3	4	5	
	0	∞	∞	∅	∞	
IT. 1	0	0	2	1	∞	✓
IT. 2	0	0	2	-2	4	✓
IT. 3	0	-3	-1	-2	4	✓
IT. 4	0	-3	-1	-5	1	✓

se mai continuă (eroare)

⇒ mai pot relaxa muchii

IT 5 : 0 -6 -4 -5 -2

IT 6 : 0 -9 -4 -11 -5

Scanned with CamScanner

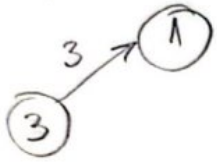
(2)

(3)

selectam 3

1	2	3	4	5	6	7
3/3	10/3	-	∞/0	∞/0	∞/0	∞/0

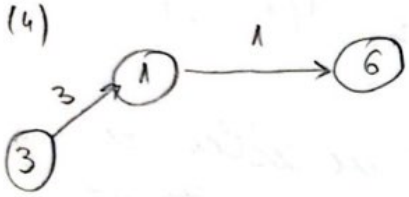
(3)



selectam 1

1	2	3	4	5	6	7
-	10/3	-	∞/0	∞/0	4/1	∞/0

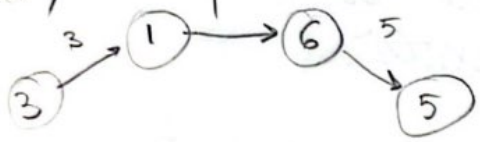
(4)



selectam 6

1	2	3	4	5	6	7
-	10/3	-	12/6	9/6	-	∞/0

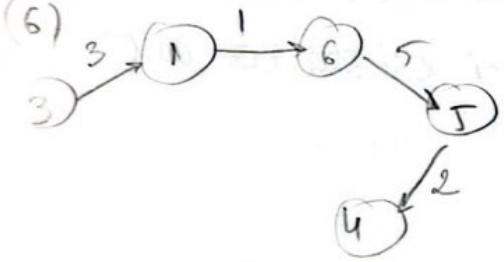
(5)



selectam 5

1	2	3	4	5	6	7
-	10/3	-	11/6	-	-	∞/0

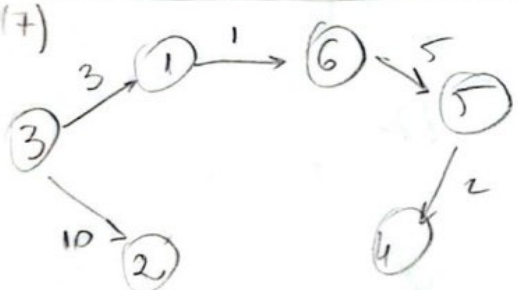
(6)



selectam 2

1	2	3	4	5	6	7
-	-	-	11/6	-	-	12/2

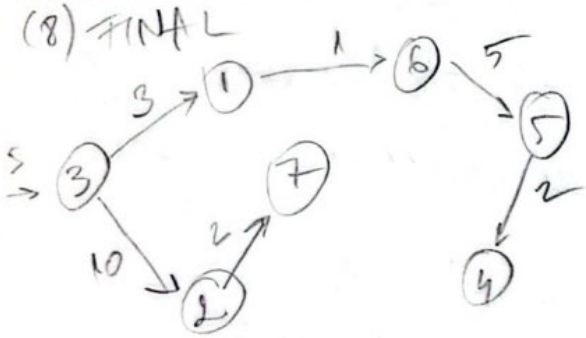
(7)



selectam 4

1	2	3	4	5	6	7
-	-	-	-	-	-	12/2

(8) FINAL



selectam 7

1	2	3	4	5	6	7
-	-	-	-	-	-	-

FINAL □

Scanned with CamScanner

modificare pseudocod (mai adăugăm un for)

...

pentru fiecare $uv \in E$ execută

dacă $d[u] + w(u,v) < d[v]$ atunci

raportează "ciclu negativ detectat"

returnează.

("dacă mai putem să relaxăm un nod", în sensul că am găsit ciclu negativ).

GRAF PLANAR

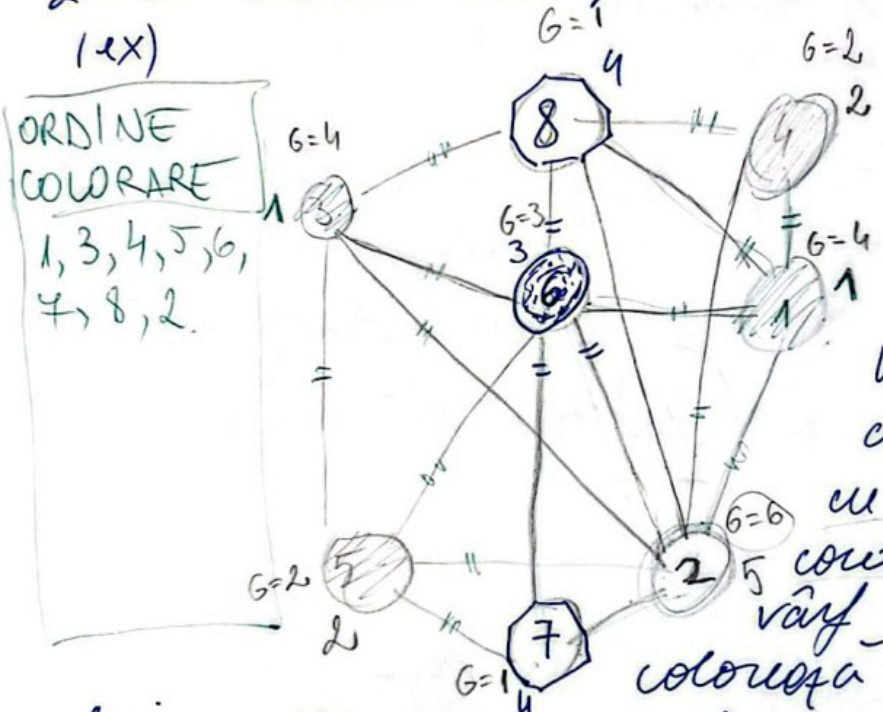
Th culor 6 culori

Un G gr. neorientat este planar $\Leftrightarrow$ admite o reprezentare în plan a.î. muchiile le corespund segmente de curbe continue care nu se intersectează în interiorul unele pe altele.

(ex)

ORDINE COLORARE

1, 3, 4, 5, 6, 7, 8, 2.

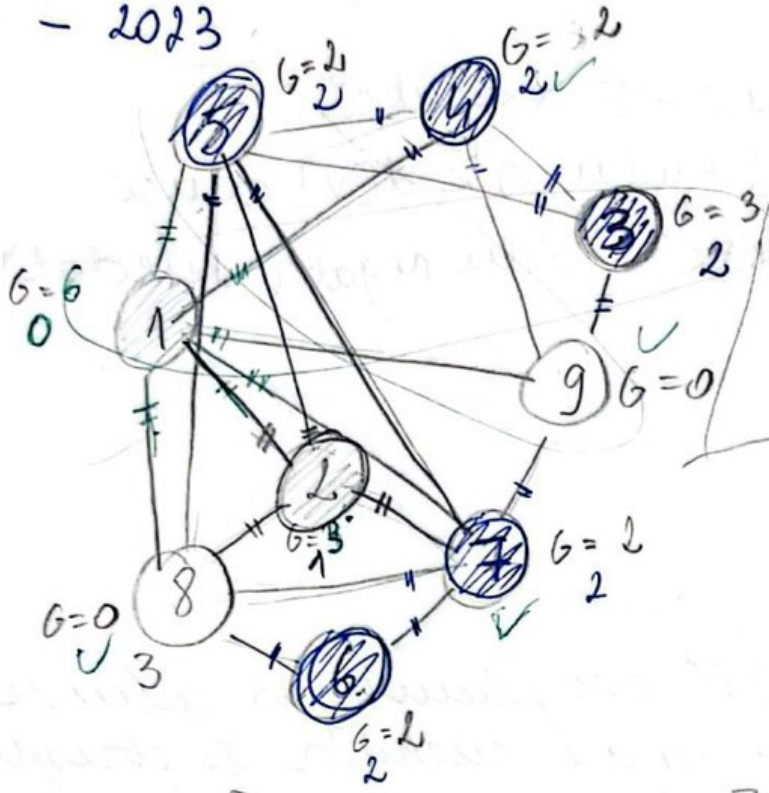


Orice graf planar și complex admite o 6-colorare.

Dacă are nu mai de vârfuri ≤ 6 atunci colorăm câte un vârf cu o culoare, iar în caz contrar, se alege un vârf cu gradul ≤ 5 și se colorează cu una din cele 6 culori pe care NU o are deja un vecin. Colorăm vârfurile crescător dacă gradul $v_f \leq 5$, altfel nodul respectiv va fi colorat la final. Gradul unui nod scade în mom. în care am colorat un nod vecin cu acesta

Scanned with CamScanner

(11) – 2023

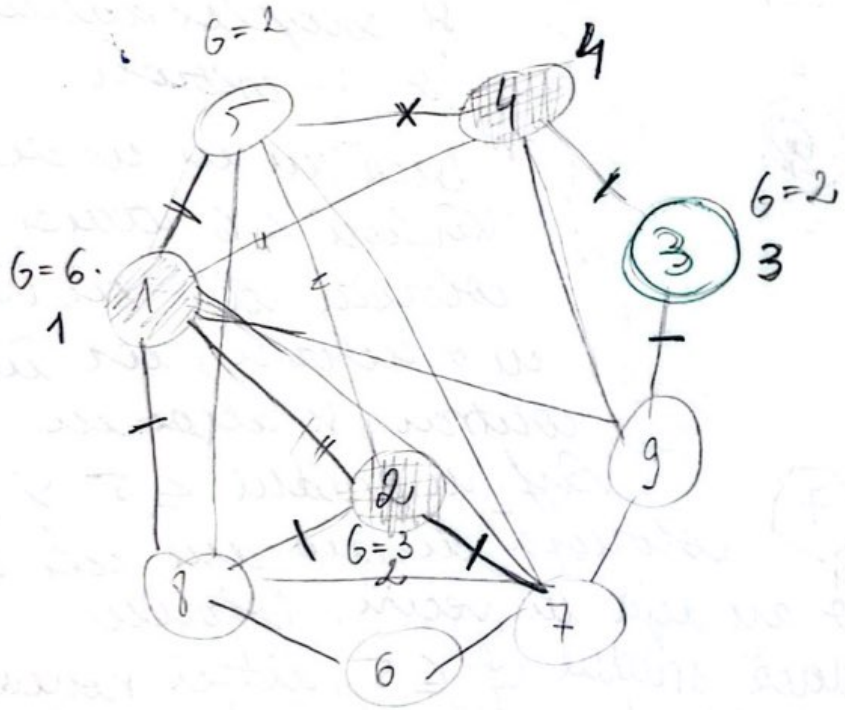


NU AM VOIE SĂ AM VÂRFURI ADIACENTE DE AC. CULOARE

CULORI = {0, 1, 2, 3, 4, 5}

ORDINE :

1	2	3	4	5	6	7	8	9
1	1	1	1	1	1	1	1	1
0	1	2	2	2	2	2	0	0



aspte
astpe
astep
astept

Scanned with CamScanner

gradul unei muchii

$d_G(u)^-$ → grad interior

$d_G(u)^+$ → grad exterior

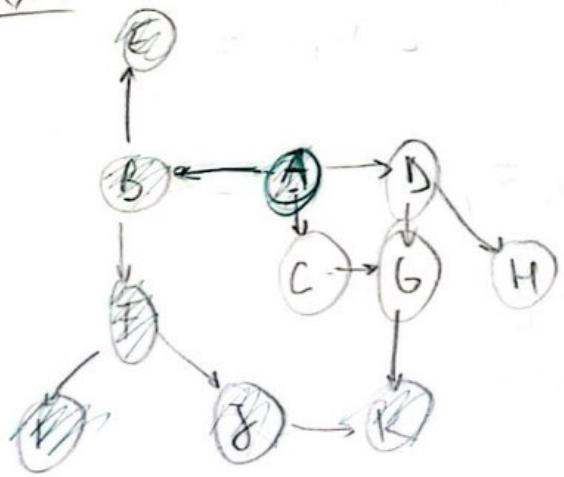
- Grafuri IZOMORFE → au aceeași structură

- Graf BIPARTIT = (∃) $P(V)$ în 2 submulțimi
  - extremitățile unui vârf al unei muchii să fie în prima mulțime de vârfuri, iar celelalte extremități de vârf în a 2-a mulțime.
  - $c(x) != c(y)$ culorile a vf cu 2 cul
  - ∄ muchii cu nodurile din aceeași submulțime.

⇒ nu pot avea cicluri de lungime impară!

- SORTARE TOPOLOGICĂ → gr. orientate!!
  - ↳ determinare de drumuri critice
  - ↳ graful admite S.T ⇔ ACICLIC (DAG)
  - ↳ implementare similară cu bf/df.

ex:



df(A)

I, K, J, F, E, B, G, C, H, D, A

⇒ inversăm

A, D, H, C, G, B, E, F, J, K, I

Scanned with CamScanner

- MUCHII CRITICE (c. neorientată) → muchie care nu e conținută în niciun ciclu.

  ↳ muchie de avansare ij critică ⇔ niv_min[j] > nivel[i]

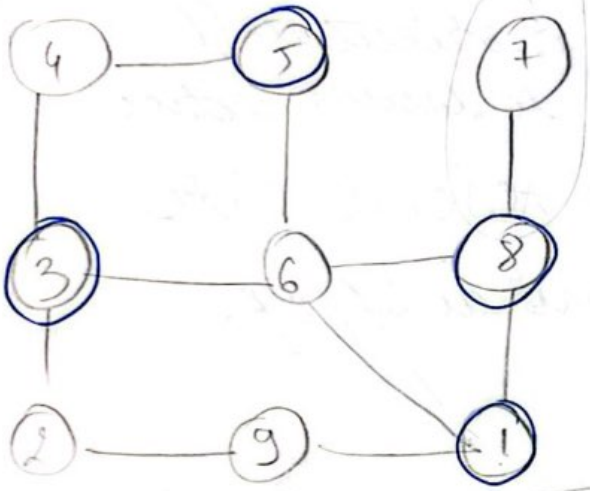
  ↳ muchie de întoarcere (nivel[j] < nivel[i] - 1) → ele închid cicluri

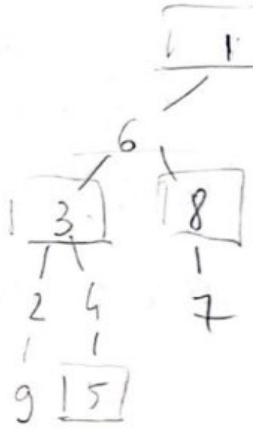
- PUNCTE CRITICE = 2 vârfuri x, y != v a.î. aparțin oricărui x-y lanț

  ⊛ ARBORE DF
  rădăcina este punct critic ⇔ are cel puțin 2 fii în arborele DF.

  ⊛ un alt vârf i din arb este critic ⇔ are cel puțin un fiu j cu niv_min[j] ≥ nivel[i]

ex:





→ mă mut la cele care fac parte dintr-un ciclu

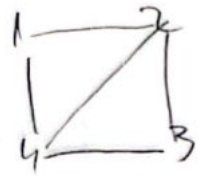
→ NODURI CRITICE: 8, 3, 5, 1.

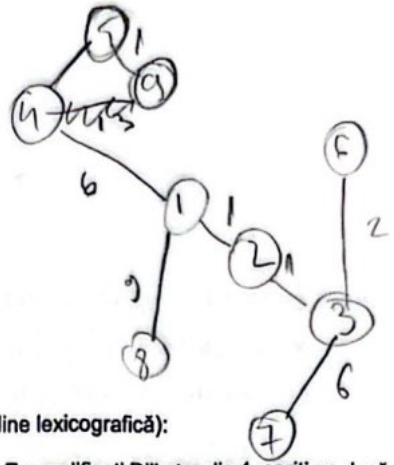
→ MUCHII CRITICE: 7-8

→ mă mut la muchiile care conțin cicluri

n=4

m=5

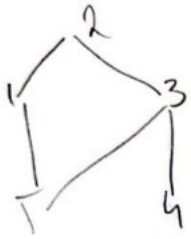


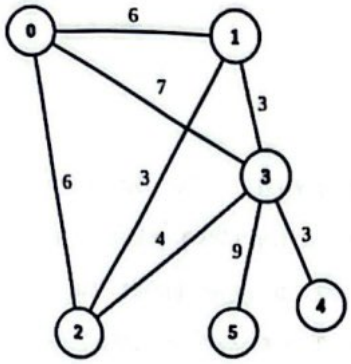


EXTRA

Pentru graful din stânga (vecinii se consideră în ordine lexicografică):

m=7





1) Exemplificați Dijkstra din 4, opriți-va după ce ați găsit distanța către 0

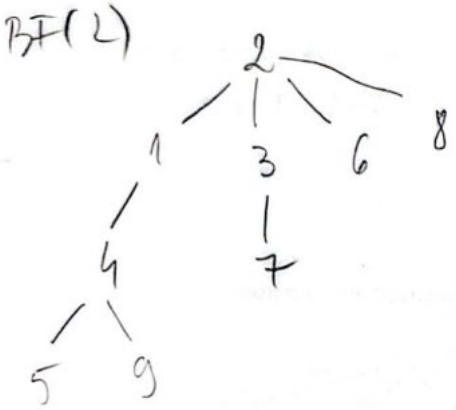
2) Cum funcționează algoritmul lui Prim din 2 ? Exemplificați alegerea primelor 6 muchii.

3) Este graful bipartit ? Dacă nu eliminați un număr minim de muchii astfel încât el sa devina bipartit. Care este numărul maxim de muchii ale unui graf bipartit cu 8 vârfuri? Justificați.

4) Există lanț eulerian în graf? Dacă nu adăugați număr minim de muchii astfel încât graful format sa admită lanț eulerian, descriind și strategia după care ați adăugat muchiile. Indicați un lanț eulerian în graful obținut. Enunțați o condiție necesară și suficientă ca un graf neorientat să aibă un lanț eulerian.

intră

BF(2)



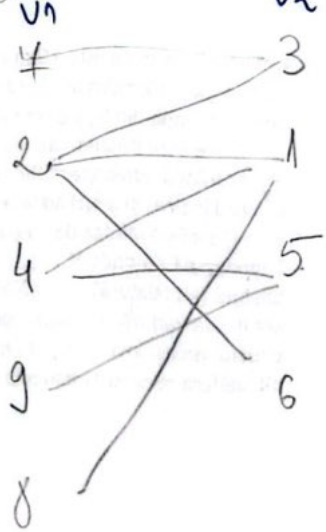
Q: 2, 1, 3, 6, 8, 4, 7, 5, 9

ies

1
1,4 +
1,0
0,5
0,5
3,5

muchia (4,9)

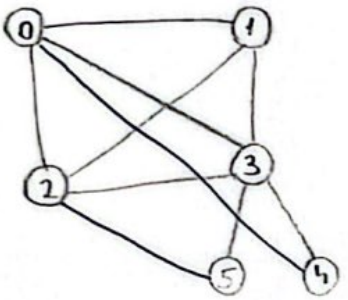
Drumuri de la nodul 7



4. condiție necesară și suficientă ca un graf neorientat să aibă un lanț eulerian: să aibă cel mult 2 vârfuri de grad impar

avem toate vârfurile de grad impar deci nu avem lanț. eulerian

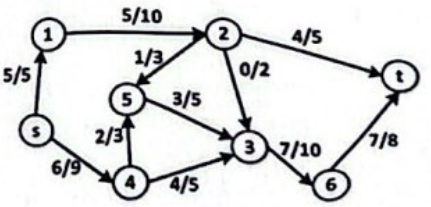
ca să adăugăm număr minim de muchii vom lăsa 2 noduri cu grad impar deci trebuie să creștem gradul cu o unitate la 4 dintre noduri => unim 2 noduri și alte 2 noduri din graf



am unit 2-5 și 0-4

lanț eulerian: 3-2-5-3-
-0-1-2-0-4-3-1

5) **Definiți noțiunile de flux, tăietură, tăietură minimă și lanț nesaturat/drum de creștere. Ilustrați pașii algoritmului Ford-Fulkerson pentru rețeaua din figura următoare (pe un arcul e sunt trecute valorile f(e)/c(e) reprezentând flux/capacitate), pornind de la fluxul indicat și alegând la fiecare pas un s-t lanț f-nesaturat de lungime minimă (algoritmul Edmonds-Karp). Indicați o tăietură (s-t tăietură) minimă în rețea (se vor indica vârfurile din bipartiție, arcele directe, arcele inverse) și determinați capacitatea acestei tăieturi. Mai există și o altă s-t tăietură minimă în această rețea? Justificați răspunsurile. (1p)**



5. Flux: un flux într-o rețea de transport $N = (G, S, T, C)$ este o funcție $f: E \to \mathbb{N}$ cu proprietățile:

1) $0 \leq f(e) \leq c(e)$, $\forall e \in E(G)$ condiția de mărginire

2) Pentru orice vârf intermediar v

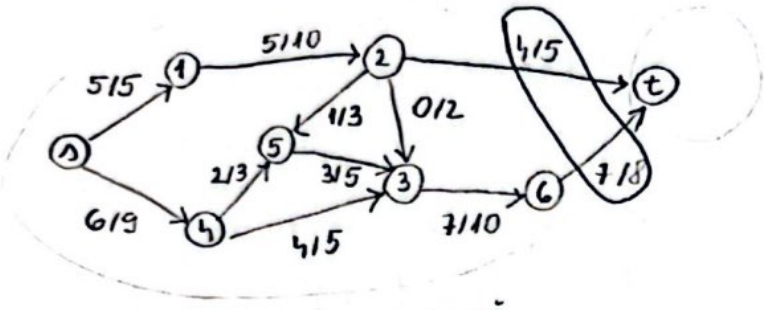
$$\sum_{uv \in E} f(uv) = \sum_{vu \in E} f(vu)$$ condiția de conservare a fluxului

Tăietură: Fie $N = (G, S, T, i, C)$ o rețea. O s-t tăietură $K = (X, Y)$ în rețea este o bipartiție (X, Y) a mulțimilor vârfurilor V astfel încât $s \in X$ și $t \in Y$

Tăietură minimă: Fie N o rețea. O tăietură $\tilde{K}$ se numește tăietură minimă în N dacă $c(\tilde{K}) = \min \{c(K) \mid K$ este tăietură în $N\}$

Lanț nesaturat: un s-t lanț P se numește lanț nesaturat sau drum de creștere dacă $i(P) \neq 0$ unde $i(P)$ = capacitatea reziduală a lanțului

Scanned with CamScanner



K = tăietură minimă

C(K) = 5 + 8 = 13

bipartiție : - s, 1, 2, 3, 4, 5, 6 (partea 1)
- t (partea 2)

nu mai există o altă tăietură minimă:

9 + 5 = 14 > 13
9 + 10 = 19 > 13
10 + 3 + 5 = 18 > 13
10 + 3 + 5 + 5 = 23 > 13
10 + 10 = 20 > 13
5 + 5 + 5 = 15 > 13
5 + 10 = 15 > 13
5 + 8 = 13 minim

nu mai există o altă tăietură minimă:

9 + 5 = 14 > 13
9 + 10 = 19 > 13
10 + 3 + 5 = 18 > 13
10 + 3 + 5 + 5 = 23 > 13
10 + 10 = 20 > 13
5 + 5 + 5 = 15 > 13
5 + 10 = 15 > 13
5 + 8 = 13 minim

Scanned with CamScanner

5) Care este distanta de editare intre cuvintele “examen” si “marire” ? Justificați

5. examen → marinc

mxamen
mamen
mar en
mari en
marinen
marinc

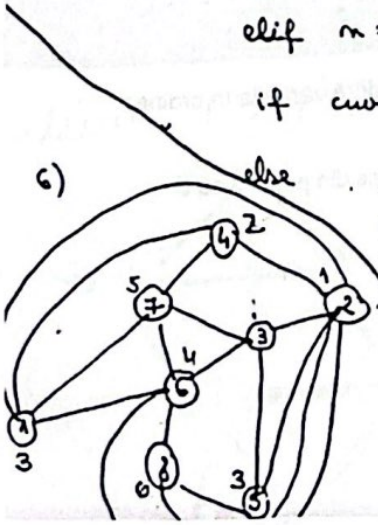
=> distanta de editare e 6

5) Care este distanta de editare între cuvintele “viata” si “restanta” ? Justificați

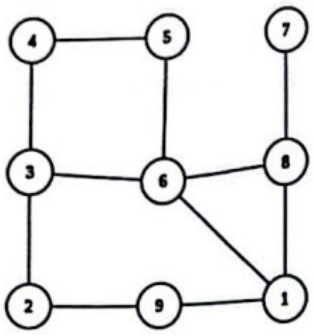
5) Distanta de editare este 5

```
dist_editare(cuv1, cuv2, n, m):
    if m==0:
        return n
    elif n==0:
        return m
    if cuv1[n] = = cuv2[m]:
        dist_edit(cuv1, cuv2, n-1, m-1)
    else:
        ins = dist_editare(cuv1, cuv2, n, m-1)
        del = dist_editare(cuv1, cuv2, n-1, m)
        repl = dist_editare(cuv1, cuv2, n-1, m-1)
        return 1+min(ins, del, repl)
```

6)



graful este planar pt. că putem să îl desenăm fără a intersecta muchii



Pentru graful din imaginea din stânga:

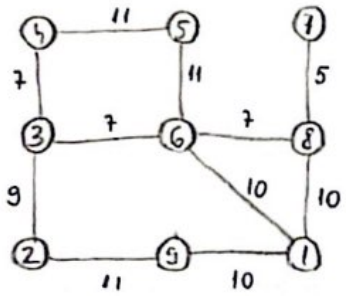
1) Care sunt nodurile critice ?

2) Care sunt muchiile critice ?

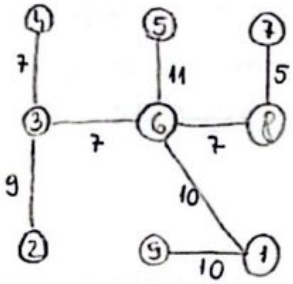
3) Exemplificati cum funcţionează bf(9) până când sunt vizitate 6 noduri, ilustrand si arborele bf asociat; vecinii unui vârf se consideră în ordine lexicografică

4) Puneţi ponderi pe muchii astfel încât costul unui arbore parţial de cost minim în graful obţinut să fie 66.

4.



apcm →



Componente tare conexe - Algoritm Kosaraju

Următorul algoritm de timp liniar (adică **θ(V+E)**) determină componentele tare conexe ale unui graf orientat G = (V, E), folosind două căutări în adâncime, una în G şi una în G^T.

Componente-Tare-Conexe(G)

1. apelează CA(G) pentru a calcula timpii de terminare f[u] pentru fiecare vârf u
2. calculează G^T
3. apelează CA(G^T), dar, în bucla principală a lui CA, consideră vârfurile în ordinea **descrescătoare** a timpilor f[u] (calculaţi la linia 1)
4. afişează vârfurile fiecărui arbore din pădurea de adâncime din pasul 3 ca o componentă tare conexă separată

Scanned with CamScanner

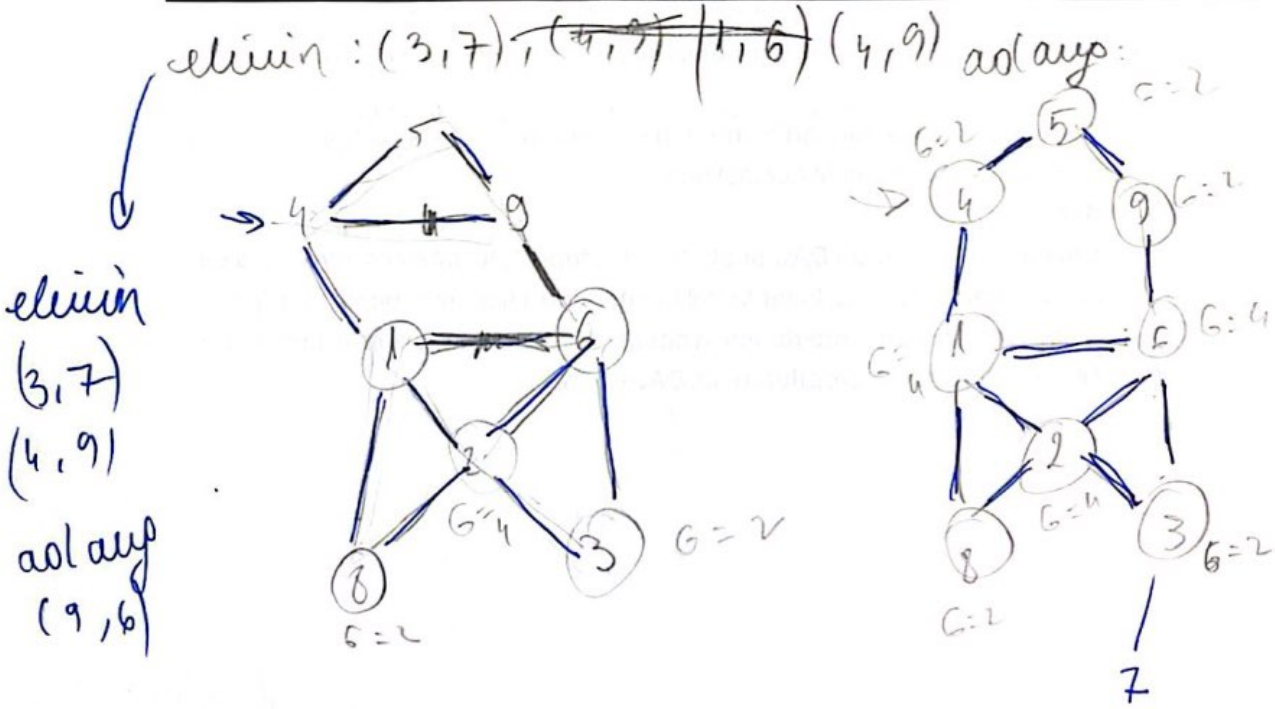
Grafuri Hamiltoniene:

- **Definiție**: Un graf hamiltonian este un graf care conține un ciclu Hamiltonian, adică un ciclu care trece prin fiecare vârf al grafului exact o dată și revine la vârful inițial.
- **Ce trebuie să știi**:
  - **Ciclul Hamiltonian**: Trece prin toate vârfurile o singură dată.
  - **Criterii**: Nu există un criteriu simplu general pentru a determina dacă un graf este hamiltonian, dar teorema lui Dirac și teorema lui Ore oferă condiții suficiente.
  - **Aplicabilitate**: Probleme de optimizare, cum ar fi problema comis-voiajorului (TSP).

Grafuri Euleriene:

- **Definiție**: Un graf eulerian este un graf care conține un ciclu Eulerian, adică un ciclu care trece prin fiecare muchie a grafului exact o dată și revine la vârful inițial.
- **Ce trebuie să știi**:
  - **Ciclul Eulerian**: Trece prin toate muchiile o singură dată.
  - **Criterii**:
    - Într-un graf neorientat, toate vârfurile trebuie să aibă grad par.
    - Într-un graf orientat, toate vârfurile trebuie să aibă gradul de intrare egal cu gradul de ieșire.
  - **Aplicații**: Probleme de trasee, cum ar fi traversarea unui pod (Problema Podurilor din Königsberg).

Pe scurt, un **graf Hamiltonian** implică un ciclu care vizitează fiecare **vârf** o dată, iar un **graf Eulerian** implică un ciclu care vizitează fiecare **muchie** o dată.



Scanned with CamScanner

Algoritmi – G=(V, E) graf orientat

G – neponderat Parcurgere lățime BF	G – ponderat, ponderi >0 Algoritmul lui Dijkstra	G – ponderat fără circuite
BF(s)	**Dijkstra(s)**	**DAGS(s)**
coada C ← ∅; adauga(s, C)	(min-heap) Q ← V {se putea incepe doar cu Q ← {s} +vector viz; v∈Q ⇔ v nevizitat}	SortTop ← sortare_topologica(G)
pentru fiecare u∈V d[u]=∞;tata[u]=viz[u]=0 viz[s]← 1; d[s] ← 0	pentru fiecare u∈V d[u] = ∞; tata[u]=0 d[s] = 0	pentru fiecare u∈V d[u] = ∞; tata[u]=0 d[s] = 0
cat timp C ≠ ∅ u ← extrage(C); pentru fiecare uv∈E daca viz[v]=0 d[v] ← d[v]+1 tata[v] ← u adauga(v, C) viz[v] ← 1	cat timp Q ≠ ∅ u = extrage(Q) vârf cu eticheta d minimă pentru fiecare uv∈E daca v∈Q si d[u]+w(u,v)<d[v] d[v] = d[u]+w(u,v) tata[v] = u repara(v,Q)	pentru fiecare u ∈ SortTop pentru fiecare uv∈E daca d[u]+w(u,v)<d[v] d[v] = d[u]+w(u,v) tata[v] = u
scrie d, tata	scrie d, tata	scrie d, tata
O(n+m)	O(m log(n)) / O(n^2)	O(n+m)

DAG (Directed Acyclic Graph) este un graf orientat care nu conține cicluri, adică nu există nicio secvență de muchii orientate care să înceapă și să se termine în același nod.

Cum funcționează un DAG:

1. **Structură:**
   - **Orientat:** Fiecare muchie are o direcție (de la un nod sursă la un nod destinație).
   - **Acilic:** Nu există cicluri, adică nu poți pleca dintr-un nod și, urmând direcția muchiilor, să te întorci la același nod.
2. **Proprietăți:**
   - **Ordine topologică:** Un DAG poate fi sortat topologic, ceea ce înseamnă că nodurile pot fi ordonate liniar astfel încât, pentru fiecare muchie *(u,v)*(u, *v)*(u,v), *uu*u apare înainte de *vw* în această ordine. Aceasta este una dintre cele mai importante proprietăți ale DAG-urilor.

Scanned with CamScanner

- **Dependenţe**: Sunt adesea folosite pentru a reprezenta dependenţele într-un sistem (de exemplu, sarcini care trebuie realizate în ordine, sau versiuni ale fişierelor în controlul sursei).

3. **Exemple de Utilizare**:
   - **Planificarea sarcinilor**: În proiecte complexe, sarcinile sunt dependente una de alta (de exemplu, nu poţi începe sarcina B înainte de a termina sarcina A).
   - **Compilare**: În lanţul de compilare, un fişier sursă poate depinde de alte fişiere care trebuie compilate anterior.
   - **Versionare Git**: Git utilizează un DAG pentru a reprezenta istoria commit-urilor, unde muchiile indică dependenţele dintre commit-uri.
4. **Algoritmi**:
   - **Sortarea topologică**: Aceasta se face de obicei folosind BFS (Breadth-First Search) sau DFS (Depth-First Search) şi asigură ordonarea nodurilor într-o secvenţă liniară conform dependenţelor.
   - **Detectarea ciclurilor**: Deşi un DAG nu are cicluri, algoritmi precum DFS sunt folosiţi pentru a verifica dacă un graf este aciclic (adică un DAG).

Cum funcţionează un DAG:

Un DAG este extrem de util în situaţii în care dependenţele şi ordinea execuţiei sunt cruciale. De exemplu:

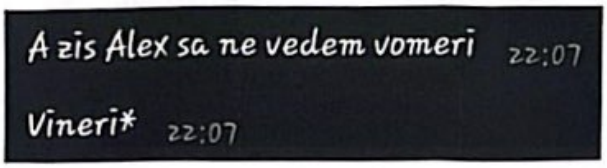
- **Într-o sarcină de planificare**, dacă sarcina A trebuie finalizată înainte de sarcina B, sarcinile sunt reprezentate ca noduri, iar dependenţa dintre ele ca o muchie orientată de la A la B.
- **Într-un proces de compilaţie**, ordinea de compilare a modulelor de cod poate fi determinată folosind un DAG pentru a respecta dependenţele dintre module.

DAG-urile sunt esenţiale în gestionarea şi organizarea structurilor complexe, unde ordinea şi dependenţele joacă un rol critic.

Distanta de editare

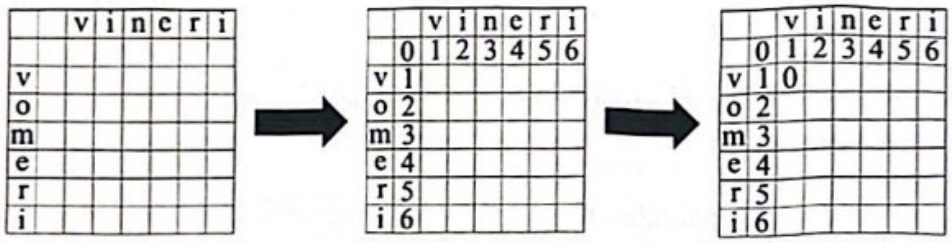
Scanned with CamScanner

Din grabă ați scris greșit cuvântul vineri. Totuși, după ce v-ați corectat greșeala, vă întrebați cât este distanța de editare între vomeri și vineri. Deci, cum o calculăm?



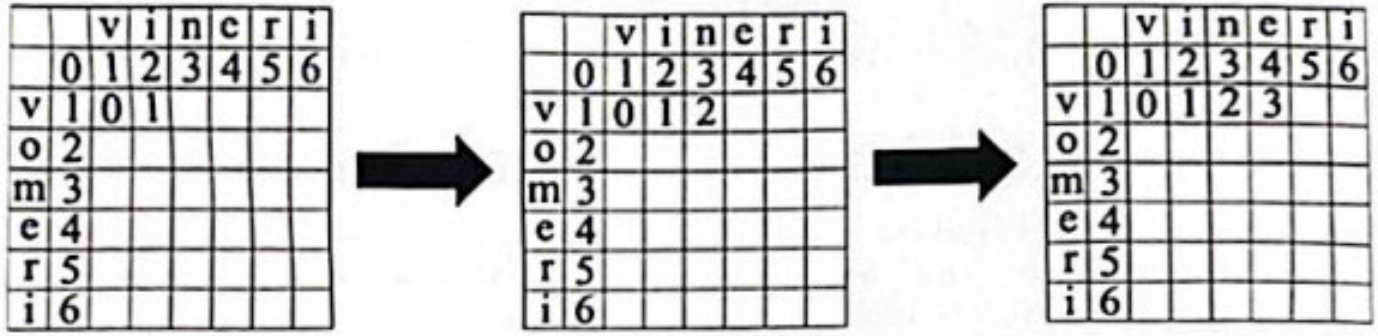
Observație: În rezolvare considerăm cuvintele având doar litere mici.

Exemplu



Scanned with CamScanner

—



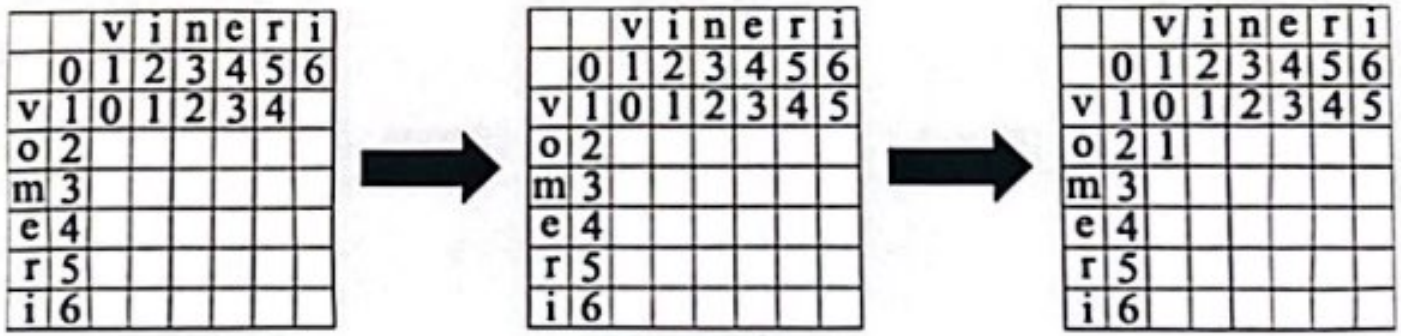
		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1				
o	2						
m	3						
e	4						
r	5						
i	6						

		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2			
o	2						
m	3						
e	4						
r	5						
i	6						

		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3		
o	2						
m	3						
e	4						
r	5						
i	6						

Exemplu

—

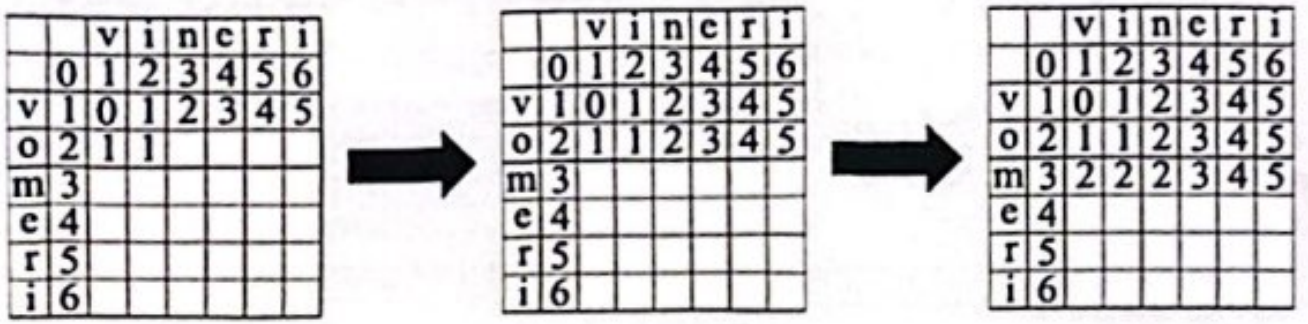


		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	
o	2						
m	3						
e	4						
r	5						
i	6						

		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2						
m	3						
e	4						
r	5						
i	6						

		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2	1					
m	3						
e	4						
r	5						
i	6						

Scanned with CamScanner

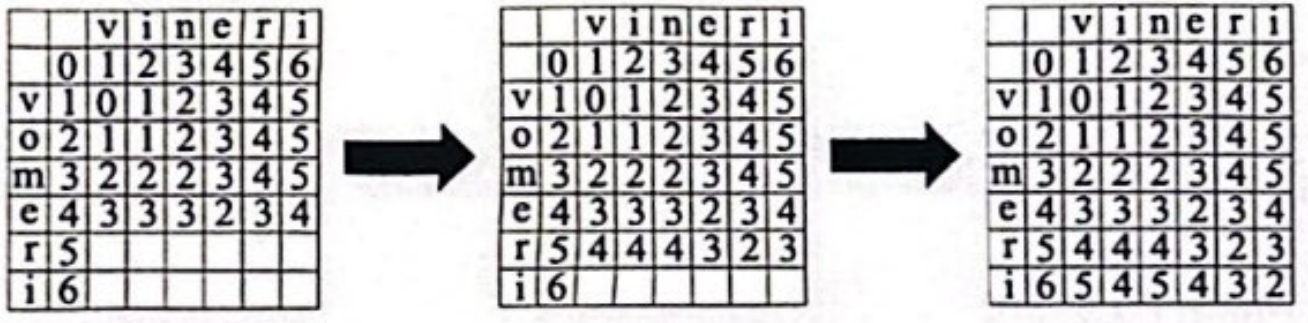


		v	i	n	c	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2	1	1				
m	3						
e	4						
r	5						
i	6						

		v	i	n	c	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2	1	1	2	3	4	5
m	3						
e	4						
r	5						
i	6						

		v	i	n	c	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2	1	1	2	3	4	5
m	3	2	2	2	3	4	5
e	4						
r	5						
i	6						

Exemplu



		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2	1	1	2	3	4	5
m	3	2	2	2	3	4	5
e	4	3	3	3	2	3	4
r	5						
i	6						

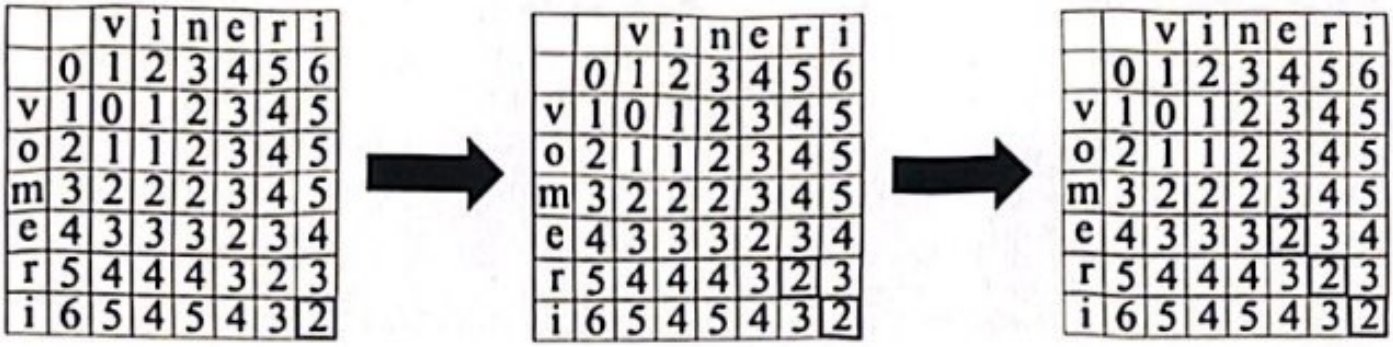
		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2	1	1	2	3	4	5
m	3	2	2	2	3	4	5
e	4	3	3	3	2	3	4
r	5	4	4	4	3	2	3
i	6						

		v	i	n	e	r	i
	0	1	2	3	4	5	6
v	1	0	1	2	3	4	5
o	2	1	1	2	3	4	5
m	3	2	2	2	3	4	5
e	4	3	3	3	2	3	4
r	5	4	4	4	3	2	3
i	6	5	4	5	4	3	2

Scanned with CamScanner

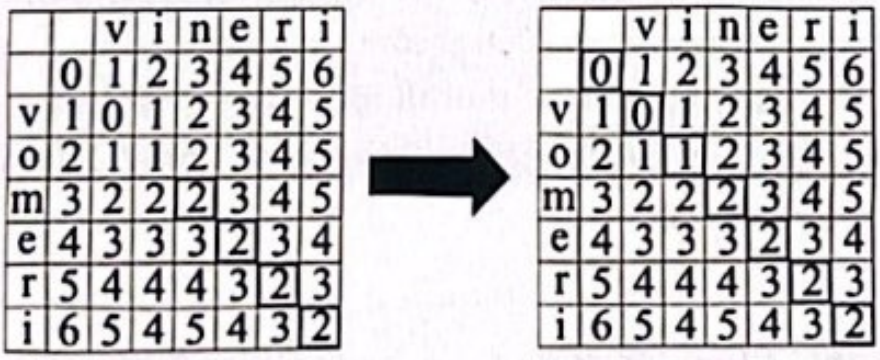
Reconstrucția pașilor

—



Reconstrucția pașilor

—



Scanned with CamScanner

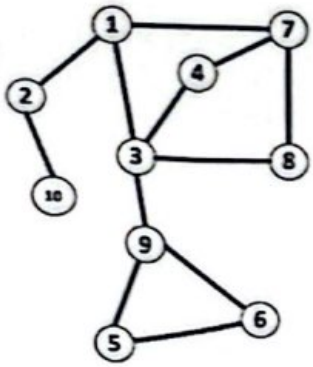
2. **Calculul valorilor din matrice**:
   - Pentru fiecare i de la 1 la m și j de la 1 la n, calculăm $D[i][j]$ pe baza valorilor vecine:

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{(stergere)} \\ D[i][j-1] + 1 & \text{(inserare)} \\ D[i-1][j-1] + cost_substituire & \text{(substituire)} \end{cases}$$

   - **Costul substituției** este 0 dacă $A[i-1] = B[j-1]$ și 1 dacă $A[i-1] \neq B[j-1]$.
3. **Valoarea finală**:
   - După ce umplem toată matricea, valoarea $D[m][n]$ va fi distanța de editare între cuvintele A și B.

EXAMEN LA ALGORITMI FUNDAMENTALI

Pentru graful din imaginea din stânga rezolvați cerințele 1-4 și justificați răspunsurile; vecinii unui vârf se consideră în ordine lexicografică



1) (0,5p) Care sunt muchiile critice (definiți și noțiunea)?
2) (0,5p) Exemplificați (cu explicații) cum funcționează parcurgerea în adâncime df(4), ilustrând si arborele df asociat
3) (0,75p) Indicați un subgraf bipartit (indus) al acestui graf cu număr maxim de noduri și indicați o bipartiție (sau bicolorare) a acestuia, explicând și cum ați construit-o.
4) (0,75p) Puneți ponderi pe muchii astfel încât graful să aibă exact 2 arbori parțiali de cost minim de cost 50 și indicați acești arbori.

---

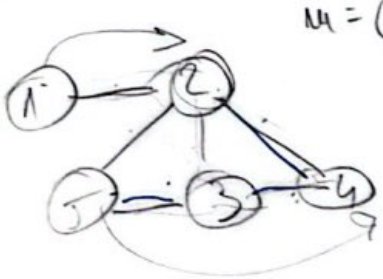
fie graful neorientat conex:



n = 6
m = 8
5 → 2

=> Df(1) = 1, 3, 5, 6, 4, 2

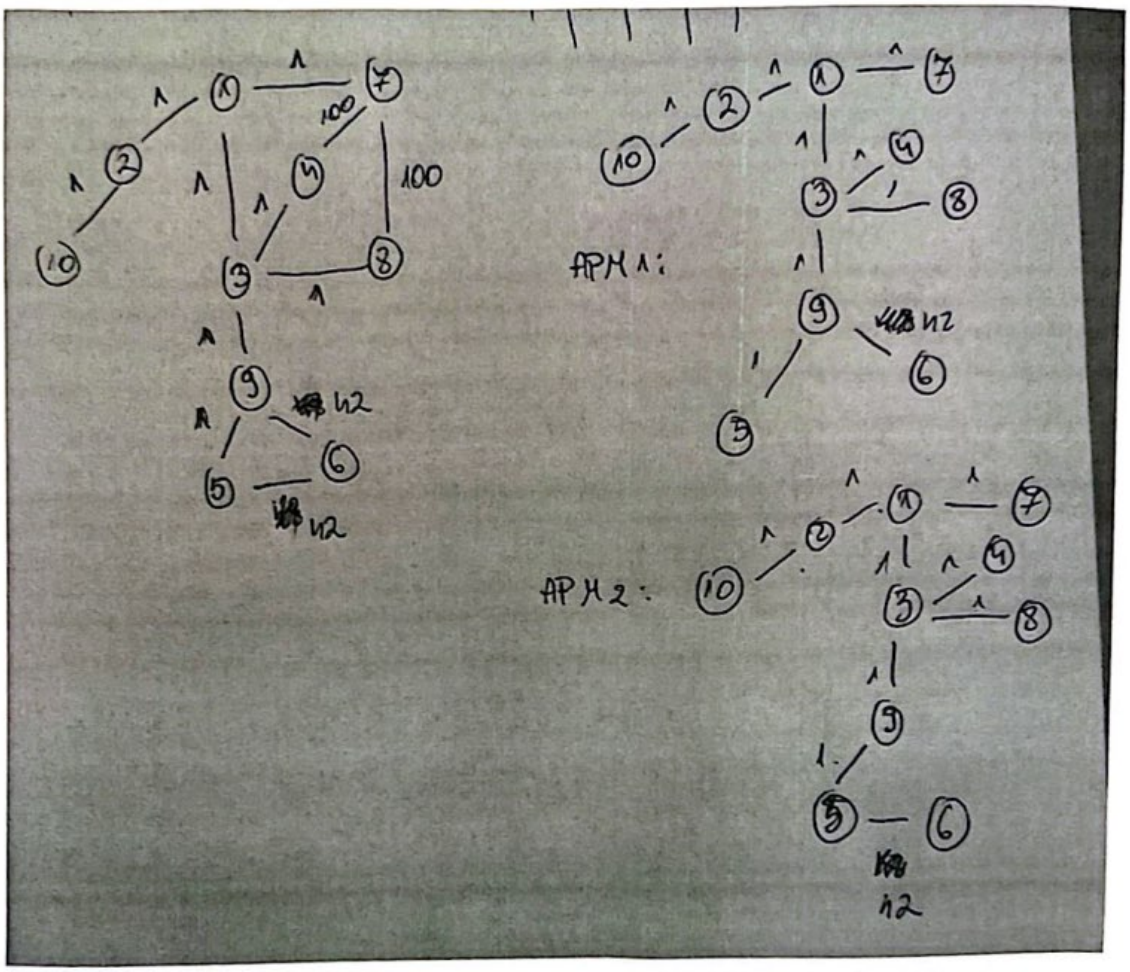
n = 5
m = 6



Df(2)



Scanned with CamScanner


100
100
APM 1:
APM 2: